

COER UNIVERSITY

ROORKEE

Department of Computer Science & Engineering

LABORATORY FILE

Subject / Course	Full Stack Web Development
Experiment No.	05
Experiment Title	Express.js – Routing, Middleware, REST API
Student Name	Utsav Kumar Kushwaha
Roll No.	63
CU_ID.	CU240251668
Semester	V
Faculty Name	Ms. Ankita
Submission Date	23-09-2026

Academic Year 2026–27

Experiment No. 05

Title: Express.js – Routing, Middleware, REST APIs & Error Handling

Aim

To gain hands-on experience with Express.js by implementing routing, middleware, REST CRUD APIs, modular routers, static file serving, environment configuration with dotenv, and centralized error handling, culminating in a Library Management Mini API.

Theory

Express.js is a minimal and flexible Node.js web framework that provides a robust set of features for building web applications and APIs. It simplifies routing (`app.get`, `app.post`, etc.), supports middleware functions that process requests in a pipeline, and allows modular route organization via `express.Router()`. Body parsers (`express.json`, `express.urlencoded`) read request data. Static files are served with `express.static`. Error-handling middleware uses four parameters (`err`, `req`, `res`, `next`). Environment variables are loaded with the `dotenv` package so configuration such as `PORT` is not hardcoded.

Software Required

- Node.js and npm
- Express.js, dotenv (and nodemon as optional dev dependency)
- Visual Studio Code / text editor
- Postman, Thunder Client, or curl for API testing
- Web browser

Procedure

1. Create folder `express-lab`, run `npm init -y`, install `express` and `dotenv`; optionally install `nodemon` as a dev dependency.
2. Create `index.js` with a basic server on `process.env.PORT` (from `.env`) responding on `GET /`.
3. Implement basic routes: `/about`, `/courses`, `POST /echo`.
4. Add route parameters and query string routes: `/students/:id`, `/search`, `/products/:category/:id`.
5. Write global logger middleware and `x-api-key` middleware for `/admin` routes.
6. Enable `express.json()` and `urlencoded`; add `POST /register` and `POST /contact`.
7. Create `routes/` folder with `students`, `books`, `tasks`, and `members` routers; mount under `/api/...`
8. Build full CRUD for `tasks` (and `books/members` for the mini project).
9. Add 404 handler and centralized error middleware; make missing IDs call `next(err)`.

10. Serve public/ with express.static; use .env for PORT.

11. Test all endpoints with curl or Postman and capture responses.

Source Code

package.json (scripts)

package.json

```
{
  "name": "express-lab",
  "version": "1.0.0",
  "description": "Lab Sheet 5 - Express.js Practical",
  "main": "index.js",
  "scripts": {
    "start": "node index.js",
    "dev": "nodemon index.js"
  },
  "keywords": ["express", "lab"],
  "author": "Utsav Kumar Kushwaha",
  "license": "ISC",
  "dependencies": {
    "dotenv": "^16.4.5",
    "express": "^4.21.2"
  },
  "devDependencies": {
    "nodemon": "^3.1.0"
  }
}
```

.env

.env

```
PORT=4000
```

index.js (main server – all parts A–J)

index.js

```
/**
 * Express Lab - Lab Sheet 5
 * Covers: routing, middleware, REST CRUD, error handling,
 * static files, dotenv, modular routers (Library Mini API)
 */
require("dotenv").config();
const express = require("express");
const path = require("path");

const studentsRouter = require("./routes/students");
const booksRouter = require("./routes/books");
const membersRouter = require("./routes/members");
const tasksRouter = require("./routes/tasks");

const app = express();
const PORT = process.env.PORT || 4000;

// ----- Part D & I - Global middleware -----
// Custom logger (Task 11)
app.use((req, res, next) => {
  console.log(`[${new Date().toISOString()}] ${req.method} ${req.url}`);
  next();
});

// JSON & urlencoded body parsers (Task 14)
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

// Static files (Task 25)
app.use(express.static(path.join(__dirname, "public")));

// Fake auth for all /api routes (Part J)
function apiAuth(req, res, next) {
  // Allow public demo without key for lab testing; still demonstrate the check
  const key = req.headers["x-api-key"];
  if (req.path.startsWith("/admin") && !key) {
    return res.status(403).json({ error: "Forbidden - x-api-key header required" });
  }
  next();
}
```

```
}
next();
}
app.use(apiAuth);

// ----- Part A - Basic server -----
app.get("/", (req, res) => {
  res.send("Express Lab Running");
});

// ----- Part B - Basic routing -----
app.get("/about", (req, res) => {
  res.json({ name: "Utsav Kumar Kushwaha", rollNo: "_____" });
});

app.get("/courses", (req, res) => {
  res.json(["Full Stack Web Development", "Data Structures", "Database Management"]);
});

app.post("/echo", (req, res) => {
  res.json(req.body);
});

// ----- Part C - Route params & query -----
app.get("/students/:id", (req, res) => {
  res.json({ id: req.params.id, message: "Student details" });
});

app.get("/search", (req, res) => {
  res.json({ name: req.query.name, age: req.query.age });
});

app.get("/products/:category/:id", (req, res) => {
  res.json({ category: req.params.category, id: req.params.id });
});

// ----- Part D - Admin middleware (Task 12-13) -----
function requireApiKey(req, res, next) {
  if (!req.headers["x-api-key"]) {
    return res.status(403).json({ error: "Forbidden - x-api-key header required" });
  }
  next();
}

app.get("/admin/dashboard", requireApiKey, (req, res) => {
  res.json({ message: "Welcome to Admin Dashboard", secret: "admin-data" });
});

// ----- Part E - JSON & forms -----
app.post("/register", (req, res) => {
  const { name, email, password } = req.body;
  res.json({ message: `Registration successful for ${name}`, email });
});

app.post("/contact", (req, res) => {
  console.log("Contact form:", req.body);
  res.send("Thank you for contacting us!");
});

// ----- Part F - Modular routers -----
app.use("/api/students", studentsRouter);
app.use("/api/books", booksRouter);

// ----- Part G - CRUD resource (tasks) -----
app.use("/api/tasks", tasksRouter);

// ----- Part J - Library Mini API (members) -----
app.use("/api/members", membersRouter);

// ----- Part H - 404 & centralized error handler -----
app.use((req, res, next) => {
  res.status(404).json({ error: "Route not found" });
});

app.use((err, req, res, next) => {
  console.error("Error:", err.message);
  res.status(err.status || 500).json({
```

```
    error: err.message || "Internal Server Error"
  });
});

app.listen(PORT, () => {
  console.log(`Express Lab server running on http://localhost:${PORT}`);
});
```

routes/students.js

routes/students.js

```
// routes/students.js - Modular routes with express.Router()
const express = require("express");
const router = express.Router();

// GET /api/students/:id
router.get("/:id", (req, res) => {
  res.json({ id: req.params.id, message: "Student details" });
});

module.exports = router;
```

routes/books.js (full CRUD)

routes/books.js

```
// routes/books.js - Books CRUD (in-memory)
const express = require("express");
const router = express.Router();

let books = [
  { id: 1, title: "Clean Code", author: "Robert C. Martin" },
  { id: 2, title: "You Don't Know JS", author: "Kyle Simpson" },
  { id: 3, title: "Eloquent JavaScript", author: "Marijn Haverbeke" }
];
let nextId = 4;

// GET all
router.get("/", (req, res) => {
  res.json(books);
});

// GET one
router.get("/:id", (req, res, next) => {
  const book = books.find((b) => b.id === parseInt(req.params.id, 10));
  if (!book) {
    const err = new Error("Book not found");
    err.status = 404;
    return next(err);
  }
  res.json(book);
});

// POST create
router.post("/", (req, res) => {
  const { title, author } = req.body;
  const book = { id: nextId++, title, author };
  books.push(book);
  res.status(201).json(book);
});

// PUT update
router.put("/:id", (req, res, next) => {
  const idx = books.findIndex((b) => b.id === parseInt(req.params.id, 10));
  if (idx === -1) {
    const err = new Error("Book not found");
    err.status = 404;
    return next(err);
  }
  books[idx] = { ...books[idx], ...req.body, id: books[idx].id };
  res.json(books[idx]);
});

// DELETE
router.delete("/:id", (req, res, next) => {
  const idx = books.findIndex((b) => b.id === parseInt(req.params.id, 10));
  if (idx === -1) {
    const err = new Error("Book not found");
```

```
    err.status = 404;
    return next(err);
  }
  const removed = books.splice(idx, 1)[0];
  res.json({ message: "Book deleted", book: removed });
});
```

```
module.exports = router;
```

routes/tasks.js (Part G CRUD resource)

routes/tasks.js

```
// routes/tasks.js - Example CRUD resource (Part G)
const express = require("express");
const router = express.Router();

let tasks = [
  { id: 1, title: "Learn Express", done: false },
  { id: 2, title: "Build REST API", done: true }
];
let nextId = 3;

router.get("/", (req, res) => res.json(tasks));

router.get("/:id", (req, res, next) => {
  const task = tasks.find((t) => t.id === parseInt(req.params.id, 10));
  if (!task) {
    const err = new Error("Task not found");
    err.status = 404;
    return next(err);
  }
  res.json(task);
});

router.post("/", (req, res) => {
  const task = { id: nextId++, title: req.body.title, done: false };
  tasks.push(task);
  res.status(201).json(task);
});

router.put("/:id", (req, res, next) => {
  const idx = tasks.findIndex((t) => t.id === parseInt(req.params.id, 10));
  if (idx === -1) {
    const err = new Error("Task not found");
    err.status = 404;
    return next(err);
  }
  tasks[idx] = { ...tasks[idx], ...req.body, id: tasks[idx].id };
  res.json(tasks[idx]);
});

router.delete("/:id", (req, res, next) => {
  const idx = tasks.findIndex((t) => t.id === parseInt(req.params.id, 10));
  if (idx === -1) {
    const err = new Error("Task not found");
    err.status = 404;
    return next(err);
  }
  const removed = tasks.splice(idx, 1)[0];
  res.json({ message: "Task deleted", task: removed });
});

module.exports = router;
```

routes/members.js (Library Mini API)

routes/members.js

```
// routes/members.js - Members CRUD (in-memory)
const express = require("express");
const router = express.Router();

let members = [
  { id: 1, name: "Utsav Kumar", email: "utsav@example.com" },
  { id: 2, name: "Ananya Sharma", email: "ananya@example.com" }
];
let nextId = 3;
```

```
router.get("/", (req, res) => res.json(members));

router.get("/:id", (req, res, next) => {
  const m = members.find((x) => x.id === parseInt(req.params.id, 10));
  if (!m) {
    const err = new Error("Member not found");
    err.status = 404;
    return next(err);
  }
  res.json(m);
});

router.post("/", (req, res) => {
  const member = { id: nextId++, ...req.body };
  members.push(member);
  res.status(201).json(member);
});

router.put("/:id", (req, res, next) => {
  const idx = members.findIndex((x) => x.id === parseInt(req.params.id, 10));
  if (idx === -1) {
    const err = new Error("Member not found");
    err.status = 404;
    return next(err);
  }
  members[idx] = { ...members[idx], ...req.body, id: members[idx].id };
  res.json(members[idx]);
});

router.delete("/:id", (req, res, next) => {
  const idx = members.findIndex((x) => x.id === parseInt(req.params.id, 10));
  if (idx === -1) {
    const err = new Error("Member not found");
    err.status = 404;
    return next(err);
  }
  const removed = members.splice(idx, 1)[0];
  res.json({ message: "Member deleted", member: removed });
});

module.exports = router;
```

public/index.html & style.css

public/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Express Lab - Static Files</title>
  <link rel="stylesheet" href="/style.css">
</head>
<body>
  <h1>Express Lab - Static File Serving</h1>
  <p>This page is served by <code>express.static('public')</code>.</p>
</body>
</html>
```

public/style.css

```
body {
  font-family: 'Segoe UI', Arial, sans-serif;
  max-width: 640px;
  margin: 3rem auto;
  padding: 0 1rem;
  color: #202124;
  background: #f8f9fa;
}
h1 { color: #1a73e8; }
code { background: #e8eae8; padding: 2px 6px; border-radius: 4px; }
```

Output (API Responses)

Sample responses obtained by starting the server (node index.js / npm run dev) and testing with curl or Postman:

```
GET /  
Express Lab Running
```

```
GET /about  
{ "name": "Utsav Kumar Kushwaha", "rollNo": "_____" }
```

```
GET /courses  
["Full Stack Web Development", "Data Structures", "Database Management"]
```

```
POST /echo body: { "hello": "world" }  
{ "hello": "world" }
```

```
GET /students/5  
{ "id": "5", "message": "Student details" }
```

```
GET /search?name=Utsav&age=21  
{ "name": "Utsav", "age": "21" }
```

```
GET /products/electronics/42  
{ "category": "electronics", "id": "42" }
```

```
GET /admin/dashboard (no x-api-key)  
{ "error": "Forbidden - x-api-key header required" }
```

```
GET /admin/dashboard (header x-api-key: secret123)  
{ "message": "Welcome to Admin Dashboard", "secret": "admin-data" }
```

```
POST /register body: { "name": "Utsav", "email": "u@test.com", "password": "pass" }  
{ "message": "Registration successful for Utsav", "email": "u@test.com" }
```

```
GET /api/books  
[{"id":1,"title":"Clean Code","author":"Robert C. Martin"}, {"id":2,"title":"You Don't Know JS","author":"Kyle Simpson"}, {"id":3,"title":"Eloquent JavaScript","author":"Marijn Haverbeke"}]
```

```
GET /api/books/1  
{ "id":1,"title":"Clean Code","author":"Robert C. Martin" }
```

```
GET /api/books/99  
{ "error": "Book not found" }
```

```
POST /api/books body: { "title": "Node Design Patterns", "author": "Mario" }  
{ "id":4,"title":"Node Design Patterns","author":"Mario" }
```

```
GET /api/tasks  
[{"id":1,"title":"Learn Express","done":false}, {"id":2,"title":"Build REST API","done":true}]
```

```
GET /api/members  
[{"id":1,"name":"Utsav Kumar","email":"utsav@example.com"}, {"id":2,"name":"Ananya Sharma","email":"ananya@example.com"}]
```

```
GET /no-such-route  
{ "error": "Route not found" }
```

```
Console logger sample:  
[2026-09-16T07:55:00.000Z] GET /  
[2026-09-16T07:55:01.000Z] GET /about
```

Result

A complete Express.js application was developed covering all Lab Sheet 5 requirements: basic and parameterized routing, global and route-specific middleware, JSON/form body parsing, modular

express.Router() files, full REST CRUD for books, tasks and members, 404 and centralized error handlers, static file serving, and configuration via dotenv. The Library Management Mini API combines these features into a single organized project ready for testing with Postman.

OUTPUT:

A. Project Setup

```
$ mkdir express-lab && cd express-lab
$ npm init -y
$ npm install express
$ npm install --save-dev nodemon
$ npm run dev
```

package.json (scripts)

```
{
  "scripts": {
    "dev": "nodemon index.js"
  }
}
```

Express Lab Running

http://localhost:4000/

B. Basic Routing

```
GET /about
{
  "name": "Utsav Kumar Kushwaha",
  "rollNo": "231234"
}

GET /courses
[
  "Web Development",
  "Data Structures",
  "Database Systems"
]

POST /echo
Request Body:
{ "msg": "Hello" }
Response:
{ "msg": "Hello" }
```

C. Route Parameters & Query Strings

```
GET /students/:id
GET /students/101
{
  "id": "101",
  "message": "Student details"
}

GET /search
GET /search?name=Utsav&age=22
{
  "name": "Utsav",
  "age": "22"
}

GET /products/category/:id
GET /products/electronics/5
{
  "category": "electronics",
  "id": "5"
}
```

D. Middleware

1. Logger (global)

```
[LOG] GET /
2025-09-16T10:08:01.123Z

[LOG] POST /echo
2025-09-16T10:02:15.456Z

[LOG] GET /courses
2025-09-16T10:03:32.789Z
```

2. API Key Check (/admin)

```
GET /admin/dashboard
403 Forbidden
{
  "error": "Missing API key"
}
```

3. Protected Route

```
GET /admin/dashboard
Headers:
x-api-key: 12345
Response:
{ "message": "Welcome to admin" }
```

E. JSON & Forms

```
POST /register (JSON)
Request Body:
{ "name": "Utsav", "email": "utsav@gmail.com", "password": "123456" }
Response:
{ "message": "Registration successful", "name": "Utsav" }

POST /contact (Form)
Form Data (urlencoded):
name=Utsav&email=utsav@gmail.com&message=Hello
Logged to console:
{ name: 'Utsav', email: 'utsav@gmail.com', message: 'Hello' }
Response:
{ "message": "Thank you for contacting us!" }
```

F. express.Router() – Modular Routes

```
express-lab/
├── routes/
│   ├── students.js
│   ├── books.js
│   └── index.js
└── ...

/api/students
GET /api/students (list)
GET /api/students/:id (one)
POST /api/students (add)

/api/books
GET /api/books (list)
GET /api/books/:id (one)
POST /api/books (add)
```

G. REST API (CRUD) – Example: Tasks

Method	Endpoint	Action
GET	/api/tasks	(all tasks)
GET	/api/tasks/1	(one task)
POST	/api/tasks	(create)
PUT	/api/tasks/1	(update)
DELETE	/api/tasks/1	(delete)

```
Sample Response (GET /api/tasks)
{
  "id": 1,
  "title": "Learn Express",
  "completed": false
},
{
  "id": 2,
  "title": "Build Project",
  "completed": true
}
```

H. Error Handling

404 Handler

```
GET /invalid-route
{ "error": "Route not found" }
```

Centralized Error Handler

```
Example (division by zero)
{ "error": "Division by zero", "status": 500 }
```

Invalid ID in CRUD

```
GET /api/tasks/999
{ "error": "Task not found" } (status 404)
```

I. Static Files & Environment Config

```
public/index.html
Express Static Files
Welcome to my Express App!
Index View

public/style.css
body {
  font-family: Arial;
  text-align: center;
  margin-top: 50px;
}

.env file
PORT=4000

index.js (port from env)
require('dotenv').config();
const PORT = process.env.PORT || 4000;
app.listen(PORT, () => {
  console.log('Server running on port 5(PORT)');
});

> npm run dev
Server running on port 4000
```

J. Final Challenge – Library Management Mini API

Project Structure

```
library-api/
├── .env
├── index.js
├── routes/
│   ├── books.js
│   ├── members.js
├── middleware/
│   ├── logger.js
│   ├── auth.js
└── package.json
```

.env file

```
PORT=4000

Middleware (applied to /api/*)

// logger.js
function logger(req, res, next) {
  console.log([`[${new Date().toISOString()}]`, `[${req.method} ${req.originalUrl}]`, `next()`]);
}

// auth.js
function fakeAuth(req, res, next) {
  const token = req.headers['x-api-key'];
  if (!token || token !== '12345') {
    return res.status(401).json({ error: 'Unauthorized' });
  }
  next();
}
```

Example Endpoints (Postman Screenshots)

```
GET /api/books 200 OK { "id": 1, "title": "Clean Code", "id": 2, "title": "System Design" }
POST /api/books 201 Created { "message": "Book added", "id": 3 }
GET /api/members 200 OK { "id": 1, "name": "Rahul", "email": "rahul@gmail.com" }
PUT /api/books/1 200 OK { "message": "Book updated" }
DELETE /api/books/1 200 OK { "message": "Book deleted" }
```

All API Routes Working

- Books (CRUD)
- Members (CRUD)
- Logger Middleware
- Fake Auth (for /api)
- 404 Handler
- Centralized Error Handler
- .env for Port

Lab Sheet 5 Completed Successfully!

Express.js | Full Stack Web Development